

期中（已更新至期末）要点

第二章 算法基础

*函数：插入排序★；归并排序★★

1. 循环不变式：每次迭代前，子数组 $A[1 \dots j - 1]$ 已排序。

证明：初始化、保持、终止。

2. 渐近紧确界： θ 符号。可简化时间复杂度表达式。

第三章 函数的增长

1. 证明 $f(n) = \theta(n)$ 或 $O(n)$ 或 $\Omega(n)$ 时，一定要“包裹”。（ $f(n)$ 处在“之间”）

2. 斯特林公式 $n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$ ， $n! = O(n^n)$ ， $\lg(n!) = n \lg n$ 。

3. 多重对数函数： $\lg^*(n)$ 表示需要多少次对数操作使结果 ≤ 1 。

第四章 分治策略

*函数：寻找跨越中点的最大子数组★；递归求解最大子数组；Strassen 算法

1. 最大子数组问题：寻找数组中连续子数组的和的最大值。

递归分析：将数组分为左、右、跨越中点三部分，递归求解左右子数组，再合并跨越中点的结果。

2. 代入法求解递归式：猜测解的形式 $\rightarrow$ 数学归纳法验证。

处理边界条件：通过调整归纳假设（如减去低阶项）解决边界矛盾。例如，若归纳失败，尝试猜测 $T(n) \leq cn - d$ （减去低阶项）。

3. 递归树方法求解递归式：每层代价求和，总代价为各层代价之和。

4. 主定理： $T(n) = aT(n/b) + f(n)$

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{if } f(n) \in O(n^{\log_b a - \epsilon}), \epsilon > 0 & (1) \\ \Theta(n^{\log_b a} \lg^{k+1} n) & \text{if } f(n) \in \Theta(n^{\log_b a} \lg^k n), k \geq 0 & (2) \\ \Theta(f(n)) & \text{if } f(n) \in \Omega(n^{\log_b a + \epsilon}), \epsilon > 0 \text{ and} & (3) \\ & af(n/b) \leq cf(n) \text{ for some} \\ & c < 1 \text{ and all sufficiently large } n \end{cases}$$

第五章 概率分析与随机算法

*函数：顺序面试并雇用更优者；随机排列应聘者后执行雇用策略；通过随机优先级生成均匀随机排列★；原址随机排列数组★★；在线雇用策略★

1. 指示器随机变量： $I(A)$ 是事件 A 的指示器变量，若 A 发生则 $I(A) = 1$ ，否则为0。因此， $E(I(A)) = P(A)$ 。
2. 随机算法：通过随机排列消除输入依赖性，保证算法期望性能一致。
3. RANDOMIZE-IN-PLACE：从当前位置到末尾随机交换元素，时间复杂度 $O(n)$ 。证明方法为循环不变式：第 i 次迭代前，每个可能的 $i - 1$ 排列概率一致。
4. 生日悖论，桶中扔球（用命中来衡量），抛硬币序列长度，在线雇用策略。

第六章 堆排序

*函数：维护子树的最大堆性质★★；建堆★★；堆排序★★；提取最大值★；增加关键字★；插入元素★

1. MAX-HEAPIFY(A, i)实现最大堆的方法（假设左右二叉树都已经是最大堆）：一路往下，只要根没有左右儿子大就对换。时间复杂度 $O(\lg n)$ 。

2. 建堆：BUILD-MAX-HEAP(A)将无序数组转换为最大堆。

思想：自底向上，从最后一个非叶节点到根节点，依次调用 MAX-HEAPIFY。

时间复杂度： $O(n)$ 。

循环不变量：每次迭代前， $i + 1$ （ i 在不断变小）到 n 的节点均为最大堆根节点。

3. 堆排序算法 HEAPSORT(A)

步骤：先建堆。时间 $O(n)$ 。

接着从 n 到2，依次不断交换 $A[n]$ 与 $A[1]$ ，将最大的元素不断放到最后，然后让堆大小减小1，再用 MAX-HEAPIFY($A, 1$)维护堆性质。时间 $O(n \lg n)$ 。

关键性质：原址排序，无需额外空间。

4. 优先队列操作

(1) HEAP-EXTRACT-MAX(A):

提取最大值 $A[1]$ ，将 $A[\text{heap-size}]$ 移至根，调用 MAX-HEAPIFY。

(2) HEAP-INCREASE-KEY(A, i, key):

若 $key > A[i]$ ，更新 $A[i]$ 并向上调整至满足堆性质。

(3) MAX-HEAP-INSERT(A, key):

插入新元素到堆末尾，再调用 HEAP-INCREASE-KEY 即可。

第七章 快速排序

*函数：划分子数组★；快速排序；随机化的划分子数组★；随机化的快速排序

1. 核心函数：PARTITION(A, p, r)将子数组 $A[p \dots r]$ 划分为两部分，左边全比 $A[r]$ 小，右边全比 $A[r]$ 大。最后返回切割位置的下标 q 。

思想：先选择主元 $x = A[r]$ 。维护指针 i 来记录最后一个小于等于 x 的元素位置)，遍历 j 从 p 到 $r - 1$ 。若 $A[j] \leq x$ ，那就交换 $A[j]$ 和 $A[i + 1]$ ，并移动 i 。

最后将主元 $A[r]$ 交换到 $i + 1$ 的位置，返回 $i + 1$ 。时间复杂度 $\theta(n)$ 。

2. 性能：平衡划分，即使划分比例为常数（如9:1），时间复杂度仍为 $O(n \lg n)$ 。

第八章 线性时间排序

*函数：计数排序★★；基数排序★；桶排序★。

1. 决策树模型：一棵完全二叉树，每个内部结点表示一次比较（如 $a_i \leq a_j$ ），叶结点表示一种可能的排列。

性质：叶结点数为 $n!$ （所有排列的可能性），树的高度表示最坏情况下的比较次数。

定理：任何比较排序的最坏时间复杂度为 $\Omega(n \lg n)$ 。证明思路：决策树高度 $\geq \log(n!) = \Omega(n \lg n)$ 。

2. 计数排序思想：（1）统计每个值的出现次数（数组 C ）；（2）将 C 修改为前缀和，以确定每个值的最终位置；（3）反向遍历输入数组，按 C 中的位置来填充最终结果。

3. 基数排序从低位开始往高位排序，否则会打乱顺序。

第九章 中位数和顺序统计量

*函数：找最小值；随机化选择第*i*小元素★；确定性选择第*i*小元素★★

1. 任何算法要同时找到最小值和最大值，至少需要 $\lceil 3n/2 \rceil - 2$ 次比较。

2. 期望为线性时间的选择算法 RANDOMIZED-SELECT(*A*, *p*, *r*, *i*)

功能：在子数组*A*[*p*...*r*]中找到第*i*小的元素。

思想：调用 RANDOMIZED-PARTITION 选择随机主元*A*[*q*]，接着递归处理：根据主元位置*k*，决定递归处理左子数组或右子数组。

时间复杂度：最坏情况 $O(n^2)$ ，期望时间 $O(n)$ 。

3. 最坏情况为线性时间的选择算法

算法步骤：(1) 分组：将 *n* 个元素分为 $\lceil n/5 \rceil$ 组，每组5个元素。

(2) 找组中位数：对每组排序后取中位数，形成中位数数组。

(3) 递归找中位数的中位数：调用 SELECT 找到中位数数组的中位数 *x*。

(4) 划分：以 *x* 为主元划分数组，确定其位置 *k*。

(5) 递归处理：根据 *i* 与 *k* 的关系，递归处理左或右子数组。

时间复杂度最坏情况： $O(n)$ ，通过分组策略保证递归规模不超过 $7n/10 + 6$ 。

第十章 栈和队列（不要求必掌握）

*函数：链表插入；链表删除；队列插入；队列删除；对象分配。

1. 栈和队列：使用数组和指针（栈顶）实现栈，要处理边界条件；使用数组和指针（队头、队尾）实现队列，要处理边界条件。

2. 链表：单链表中每个结点包含 *key* 和 *next* 指针，双向链表中还包含 *prev* 指针，循环链表中尾结点的 *next* 指向头结点，头结点的 *prev* 指向尾结点。

3. 自由表管理：支持从自由表分配空闲对象，或释放对象并加入自由表。

4. 左孩子右兄弟表示法：每个结点的信息包含：*parent*、*left-child*（最左孩子）和 *right-sibling*（它的右侧兄弟）。适用于孩子数不限的树。

第十一章 散列表

*函数：直接寻址表中的插入、查找、删除；链接法中的插入、查找、删除。

1. 链接法：每个槽位维护链表，装载因子 $\alpha = \frac{n}{m}$ 表示每个链平均存储元素数

量。查找一个元素，无论成功与否，期望时间代价都是 $O(1 + \alpha)$ 。

可采取的散列函数有：

(1) 除法： $h(k) = k \bmod m$ (m 取接近2的幂的素数)

(2) 乘法： $h(k) = \lfloor m(kA \bmod 1) \rfloor$ ，这里的 kA 可能带小数， A 常取黄金比例。

(3) 全域：随机选择散列函数。如 $h_{ab}(k) = ((ak + b) \bmod p) \bmod m$ ，可保证冲突概率很低。

2. 开放寻址法：所有元素存放在散列表中，通过探查序列解决冲突。探查方法：线性探查、二次探查、双重散列。不成功查找的期望探查次数为 $\frac{1}{1-\alpha}$ ，成功查找的探查次数为 $\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$ 。

开放寻址法中，探查槽位的计算方式被表示为 $h(k, i)$ (k 为要插入的数值)：

(1) 线性探查： $h(k, i) = (h'(k) + i) \bmod m$ ；

(2) 二次探查： $h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$ ；

(3) 双重散列： $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$ ，其中 $h_2(k)$ 应与 m 互素。

3. 完全散列：针对静态关键字集合（即关键字不再变化），通过两级散列实现最坏 $O(1)$ 时间的查找。

(1) 第一级：用全域散列函数 h 将 n 个关键字分配到 m 个槽（通常 $m = n$ ）；

(2) 第二级：对每个槽 j 中的 n 个关键字，用大小为 $m_j = n_j^2$ 的散列表，并选择一个无冲突的散列函数 h_j 。

关键性质：(1) 第二级散列表大小为平方级别，则随机选 h_j 时冲突概率小于 $\frac{1}{2}$ ，

可通过多次尝试快速找到无冲突的 h_j ；(2) 虽然第二级散列表大小为平方级别，但总空间期望仍为 $O(n)$ 。

第十二章 二叉搜索树

*函数：中序遍历★；递归查找关键字；迭代查找关键字；查找子树的最小/最大结点；查找结点的后继/前驱★；插入；删除★；移植（TRANSPLANT）★

1. 前驱与后继：

(1) 后继：右子树的最小结点，或最近的左祖先（它自身为左，祖先居右）；

(2) 前驱：左子树的最大结点，或最近的右祖先（它自身为右，祖先居左）。

2. 插入：直接一路往下检查，插入在合适位置即可。

3. 删除：有四种情况。(1) 如果没有孩子，那么直接删；(2) 如果有一个孩子，那么父死子继；(3) 如果有两个孩子，且右孩子就是后继（右孩子没有左子树），那么右孩子直接顶替父亲；(4) 如果有两个孩子，且右孩子不是后继，那么找到后继（一定是右子树的最左下），让它顶替该节点，并让它的右子树顶替它（已经是最左下了，不会有左子树）。

第十三章 红黑树

*函数：左旋、右旋★；插入★★；删除。

1. 概念：平衡二叉搜索树，通过以下 5 条性质保证树高为 $O(\lg n)$ 。

(1) 每个结点是红色或黑色；

(2) 根结点是黑色；

(3) 叶结点（NIL）是黑色；

(4) 红色结点的子结点必须是黑色（不能有连续红结点）；

(5) 从任意结点到其所有叶结点的路径上，黑色结点数相同（黑高相同）。

2. 基本性质：

(1) 黑高：表示结点 x 到叶结点的路径中黑色结点数（不含 x ）。

(2) 高度上界： $2\lg(n+1)$ 。

3. 插入：将结点从顶部遍历，插入树中，初始为红色。但是，如果这个结点的父亲是红色，那么就违反了要求。分三种情况：

(1) 叔结点为红色：那么父亲、叔叔都是红色，祖父必为黑色。把父亲、叔叔、祖父全部变色，如果祖父又和曾祖父红色冲突，那么就递归。

(2) 叔结点为黑色，且当前结点与父亲、祖父在折线上：那么对父亲进行旋转，使得当前结点、父亲与祖父在直线上。进入情况三。

(3) 叔结点为黑色，且当前结点与父亲（红）、祖父（黑）在直线上：那么对祖父进行旋转，让父亲往上，祖父往下，接着父亲、祖父颜色对调。

第十五章 动态规划

*函数：钢条切割的最大收益★；矩阵链乘法的最优括号化方案★；两个序列的最长公共子序列长度★；构造最优二叉搜索树★。

1. 原理：

(1) 最优子结构：问题的最优解包含子问题的最优解。

(2) 重叠子问题：不同子问题可能重复出现，需存储结果避免重复计算。

2. 矩阵链乘法：给定矩阵链，找到括号化方案使标量乘法次数最少。

动态规划解法 (MATRIX-CHAIN-ORDER)：填充表格 $m[i][j]$ 和 $s[i][j]$ ，时间复杂度 $O(n^3)$ 。三重循环按子问题链长递增计算，保存分割点 k 。 s 是辅助表，用来记录 $m[i][j]$ 中间的切割位置。实现时，将矩阵长度 l 从小到大遍历（保证前面问题是后面问题的子问题），然后再固定左侧右侧（即移动“窗口”）遍历一遍，最内侧寻找切割在哪里最优，再遍历一遍。

3. LCS 问题： $c[i][j]$ 表示 X_i 与 Y_j 的最长公共子序列。 $b[i][j]$ 存储回溯的箭头，表示最优解来自哪里。实际可以不需要 b 数组，根据具体情况直接往回回溯即可。

4. 最优二叉搜索树：给定关键字序列 K （已排序好，也就是说 $k_1、k_2、\dots、k_n$ 的从左到右的顺序是不能变的）的搜索频率和伪关键字的搜索概率，构造期望搜索代价最小的二叉搜索树。

核心公式： $w[i][j] = \sum_{l=i}^j p_l + \sum_{l=i-1}^j q_l$ 。（可递归）

$e[i][j] = e[i][r-1] + e[r+1][j] + w[i][j]$ 的最小值，其中 $i \leq r \leq j$ 。（相当于每个以 r 为根，其余左右子树中每个结点往下一层）

第十六章 贪心算法

*函数：活动选择问题★★；赫夫曼编码★★；拟阵 GREEDY 算法

1. 原理：

(1) 贪心选择性质：通过局部最优选择构造全局最优解，无需依赖子问题的解（对比动态规划）；

(2) 最优子结构：问题的最优解包含子问题的最优解。

2. 赫夫曼编码正确性证明的关键一步：交换性质——最优前缀码中频率最低的

两个字符是兄弟叶结点。

第十七章 摊还分析

*函数：二进制计数器自增★；动态表插入与删除★

1. 聚合法：计算一个操作序列的总时间 $T(n)$ ，然后求每个操作的平均代价，例如二进制计数器自增。

2. 核算法：为每个操作分配一个“摊还代价”，要求总摊还代价 $\geq$ 总实际代价。

步骤：为每个操作分配摊还代价，再证明信用池始终非负。例如压栈弹栈。

3. 势能法：通过势能函数 $\Phi(D)$ 表示数据结构 D 的状态。第 i 个操作的摊还代价 $\hat{c}_i$ 用势函数 Φ 可定义为： $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$ 。（实际代价加上此操作引起的势能变化）

关键要求：（1）保证初始状态 $\Phi(D_0) = 0$ ；（2）保证 $\Phi(D_i) \geq 0$ （这样， $\Phi(D_n) \geq \Phi(D_0)$ ，即可保证最终 $\hat{c}_i$ 为一个上界）；（3）设法使每个 $\hat{c}_i$ 为 $O(1)$ 。

4. 采用势能法设计动态表：
$$\Phi(T) = \begin{cases} 2 \times \text{当前元素数} - \text{表容量}, & \alpha(T) \geq \frac{1}{2} \\ \frac{\text{表容量}}{2} - \text{当前元素数}, & \alpha(T) < \frac{1}{2} \end{cases}$$

要注意，仅当 $\alpha(T)$ 不足 $\frac{1}{4}$ 时，才将表规模减半。

第二十三章 最小生成树

*函数：GENERIC-MST（贪心选择安全边，逐步扩展）；Kruskal 算法★★；Prim 算法★★

1. Kruskal 算法：边的思想。按边权从小到大遍历边集，如果当前边连接两个不同连通分量，就加入树中。时间复杂度 $O(E \lg E + E \alpha(V))$ （ $E \alpha(V)$ 为并查集操作， $\alpha(V)$ 近似于1，一般可忽略不计），适用于稀疏图。

2. Prim 算法：点的思想。任选根结点，每次选择key最小的结点加入树中，并更新邻接结点的key（表示到当前树的“距离”，一开始设为 ∞ ）。适用于稠密图，时间复杂度 $O(E + V \lg V)$ （斐波那契堆）。

第二十四章 单源最短路径

*函数：初始化；松弛；BELLMAN-FORD★★；DAG 拓扑排序；得克特拉算法★★

1. Bellman-Ford 算法：可处理带负权边的图，检测负权环，求解最短路径。

(1) 初始化所有节点的最短路径为 ∞ ，源结点为0；

(2) 进行 $|V| - 1$ 次如下操作：对每条边进行松弛；

(3) 最后检查是否存在可继续松弛的边，判断是否存在负权环。同时，如果不存在负权环，则得到的结果就是最短路径。

2. 有向无环图 (DAG) 中计算单源最短路径时，通过拓扑排序+顺序松弛的方法可以达到 $O(V + E)$ 的时间复杂度，比常规算法更高效。拓扑顺序保证了松弛操作的无后效性：处理顶点时，其所有入边已处理完毕，后续无需重复调整。

(1) 拓扑排序得到顶点的线性序列，保证对于任意边 u 总位于 v 前；

(2) 按拓扑顺序遍历每个顶点 u ，对其所有出边 (u, v) 执行松弛操作。

3. Dijkstra 算法：适用于非负权图。采用贪心策略，每次选择当前最短路径的结点加入集合，并对这个结点的邻接边进行松弛。时间复杂度为 $O(E \lg V)$ ，采用斐波那契堆可优化到 $O(V \lg V + E)$ 。

4. 差分约束核心思想：将 $x_j - x_i \leq k$ 转换为边 (i, j) 权重为 k 。添加虚拟源点 v_0 ，连接所有节点，权重为0。用 Bellman-Ford 算法求解，若存在解（无负环）则每个结点的 d 即为答案。

第二十五章 所有结点对的最短路径问题

*算法：动态规划求解最短路径权重★；Floyd 算法★★；Johnson 算法★★

1. 要求：处理可能有负权（但无负环）的图，最后得到每两个结点间最短路。

2. 动态规划方法：通过路径边数递增逐步求解最短路径权重。定义 $l_{ij}^{(m)}$ 为：从 i 到 j 的最多包含 m 条边的最短路径权重，如此即可使用动态规划。时间复杂度 $O(n^4)$ ，但若用重复平方优化（计算 2^k 直到 $2^k \geq n - 1$ ），可优化到 $O(n^3 \lg n)$ 。

3. Floyd 算法：基于中间结点集合的动态规划。时间复杂度 $O(V^3)$ 。

4. Johnson 算法（用于稀疏图，Floyd 太慢了）：核心思想为，通过重新赋权将负权边转为非负权，再用 Dijkstra 求解所有点对最短路径，从而更高效。（斐波那契堆下，时间复杂度 $O(V^2 \lg V + VE)$ ）

- (1) 新增虚结点 s ：向所有结点添加权重为0的边，保证图连通；
- (2) 用 Bellman-Ford 检测负环同时计算，如果存在负环直接报错；
- (3) 对每条边进行重新赋权： $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$ ，保证非负；
- (4) 进行 Dijkstra（稀疏图下多次进行比 Floyd 更高效）；最后还原真实权重。

第二十六章 最大流

*算法：在残存网络中找增广路径并增广流（Ford-Fulkerson）★★；Edmonds-Karp 对算法的改进

1. 流网络关键性质：容量限制；流量守恒（进等于出，除 s 和 t 外）。
2. 常见思想：多源/汇转换：添加超级源点 s^* 连接所有源点，超级汇点 t^* 连接所有汇点，新边容量设为 ∞ 。
3. Ford-Fulkerson 方法：一般情况时间复杂度为 $O(E|f^*|)$ （ f 为最大流）

(1) 残存网络 G_f ：

$$\text{残存容量 } c_f(u, v) = \begin{cases} c(u, v) - f(u, v), & (u, v) \in E \\ f(u, v), & (v, u) \in E \end{cases}$$

残存边集 $E_f = \{(u, v) | c_f(u, v) > 0\}$ （含有反向边）。

(2) 构造增广路径 G_f 中 $s \rightarrow t$ 路径 p ，残存容量 $c_f(p) = \min\{c_f(u, v) | (u, v) \in p\}$ 。

(3) 对于找到的增广路，如果边在 E 中就加上 $c_f(p)$ ，否则减去，抵消反向流。

4. Edmonds-Karp 对 Ford-Fulkerson 的改进：

Ford-Fulkerson 存在缺陷：当增广路径选择不当时，算法可能要进行 $O(|f|)$ 次迭代，其中 $|f|$ 为最大流值。

使用 BFS 寻找最短增广路径（按边数计算的最短路径）。保证了每次增广后残存网络的最短路径长度单调递增，迭代次数为 $O(VE)$ ，时间复杂度则优化到了 $O(VE^2)$ 。

5. 最大二分匹配问题：将二分图添加 s 连接所有 L 部分结点， t 连接所有 R 部分结点，所有边容量均为1。构造最大流 f ，则最大二分匹配边数 $|M| = |f|$ 。

*推送-重贴标签算法/前置重贴标签算法：可优化到 $O(V^3)$ 。